ECE569 — Homework 4

Histogram Generation: Architectural Analysis on Tesla P100

Joel Maldonado
University of Arizona, Spring 2026

Introduction

In this lab we set up to compare different implementations of histogram generation on GPU. The histogram is a fundamental data structure that counts the frequency of appearances of a certain data value in a dataset, commonly used to estimate the probability distribution of such data. Histograms find applications in computer vision, data analytics and scientific computing. The fundamental challenge of histogram generation on GPU is that when multiple threads attempt to update the same memory location (bin) a memory conflict may occur. Avoiding this conflicts are the main optimization ideas we will explore in this lab, followed by efficient data communication and probably efficient encoding.

In this Lab we have coded multiple implementations of histogram generation, from a global memory version with atomic operations (V0), to a shared memory block-privatized version (V1), to a vectorized $\mathcal{R}$=2-per-block version with RLE compression (V2, the GOAT), to an ablation of V2 without RLE (V3), to a warp-aggregated version (V4), to a bin-centric gather version with zero atomics (V5), to a sort-and-reduce approach via Thrust (V6), and finally to a multi-split hierarchical bucketing version (V7).

Table 1: Histogram kernel version summary

Version	Name	Core Pattern	Atomic Scope	Memory Access
V0	Baseline	Global Scatter	Global (N times)	32-bit coalesced
V1	Privatized	Block Privatization	Shared (N times)	32-bit coalesced
V2	The GOAT	$\mathcal{R}$=2 + RLE + `uint4`	Shared (compressed runs)	128-bit `uint4`
V3	Ablation	$\mathcal{R}$=2 + `uint4`	Shared (N times)	128-bit `uint4`
V4	Warp Aggregation	Spatial Compression	Shared (max $N/32$)	32-bit coalesced
V5	Gather	Bin-Centric Scan	None (zero)	Collaborative tiling
V6	Sort & Reduce	`thrust::sort` + reduce	None (post-sort)	Library managed
V7	Multi-split	Hierarchical Bucketing	Shared (N times)	32-bit coalesced

About the optimizations

There are three tiers of optimization strategies.

1 Experiment 1: Random Data (Dataset 6)

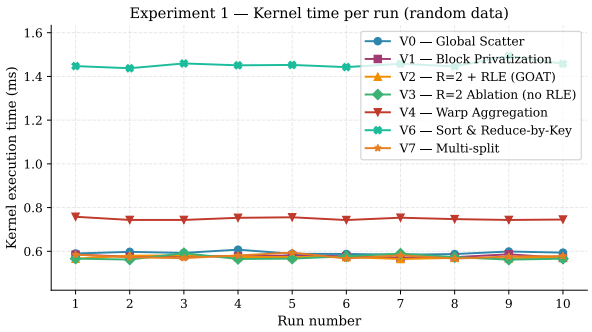
Dataset 6 contains 500,000 elements drawn uniformly at random from [0, 4095], producing a nearly flat histogram across all 4096 bins. Each kernel version was executed 10 times; kernel execution time (excluding host–device data transfer) is recorded in milliseconds.

Table 2: Kernel execution time — Experiment 1 (random, 500k elements, 4096 bins, clipped at 127)

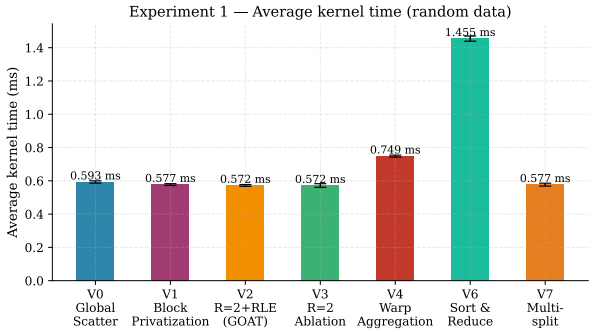
Run	V0 (ms)	V1 (ms)	V2 (ms)	V3 (ms)
1	0.5900	0.5847	0.5663	0.5663
2	0.5974	0.5755	0.5801	0.5621
3	0.5926	0.5790	0.5808	0.5912
4	0.6074	0.5769	0.5739	0.5648
5	0.5886	0.5810	0.5739	0.5664
6	0.5877	0.5774	0.5734	0.5773
7	0.5845	0.5709	0.5647	0.5893
8	0.5873	0.5732	0.5698	0.5735
9	0.5990	0.5862	0.5677	0.5618
10	0.5939	0.5697	0.5728	0.5662
Avg	**0.5929**	**0.5774**	**0.5723**	**0.5719**
Stdev	**0.0069**	**0.0055**	**0.0054**	**0.0108**

Table 3: Kernel execution time — Experiment 1, V4–V7 (random, 500k elements, 4096 bins, clipped at 127)

Run	V4 (ms)	V5 (ms)	V6 (ms)	V7 (ms)
1	0.7579	11.9899	1.4472	0.5847
2	0.7434	11.9961	1.4374	0.5725
3	0.7435	11.9959	1.4594	0.5688
4	0.7532	11.9982	1.4508	0.5809
5	0.7556	11.9826	1.4525	0.5940
6	0.7430	11.9950	1.4426	0.5671
7	0.7537	11.9871	1.4578	0.5815
8	0.7472	11.9991	1.4458	0.5667
9	0.7434	11.9848	1.4939	0.5761
10	0.7452	11.9822	1.4577	0.5781
Avg	**0.7486**	**11.9911**	**1.4545**	**0.5770**
Stdev	**0.0058**	**0.0065**	**0.0156**	**0.0087**



(a) Run-by-run timing



(b) Average with std dev

Figure 1: Experiment 1 — kernel execution time, random data

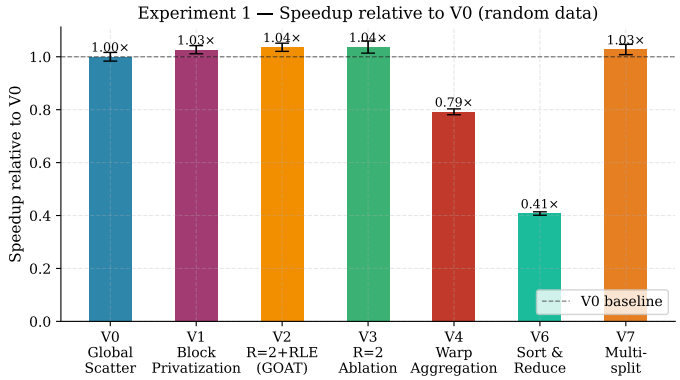


Figure 2: Experiment 1 — speedup of V1, V2, V3 relative to V0 baseline

2 Experiment 2: Uniform Data (Same Value)

The uniform dataset contains 500,000 elements all set to the same value (42), directing all atomic traffic to a single bin. This stress-tests contention in each implementation.

Table 4: Kernel execution time — Experiment 2 (uniform, 500k elements, 4096 bins, clipped at 127)

Run	V0 (ms)	V1 (ms)	V2 (ms)	V3 (ms)
1	0.9906	0.5707	0.5683	0.5654
2	0.9986	0.5744	0.5620	0.5766
3	0.9867	0.5714	0.5756	0.5764
4	0.9933	0.5980	0.5792	0.5731
5	0.9936	0.5834	0.5796	0.5647
6	0.9910	0.5732	0.5766	0.5675
7	0.9938	0.5883	0.5796	0.5636
8	0.9828	0.5904	0.5751	0.5789
9	0.9947	0.5704	0.5766	0.5770
10	1.0215	0.5865	0.5712	0.5741
Avg	**0.9947**	**0.5806**	**0.5744**	**0.5717**
Stdev	**0.0104**	**0.0099**	**0.0057**	**0.0058**

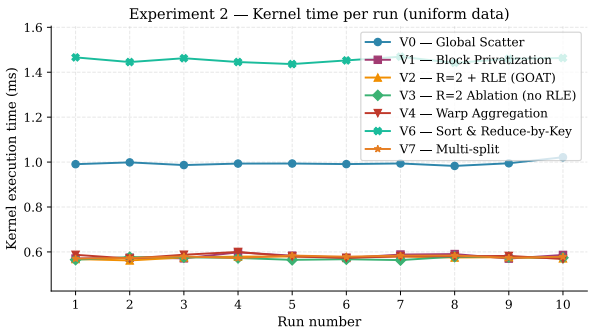
3 Analysis 1: V0 vs. V1 — Global vs. Shared Memory

The "Too Small" Dataset and the P100 L2 Cache

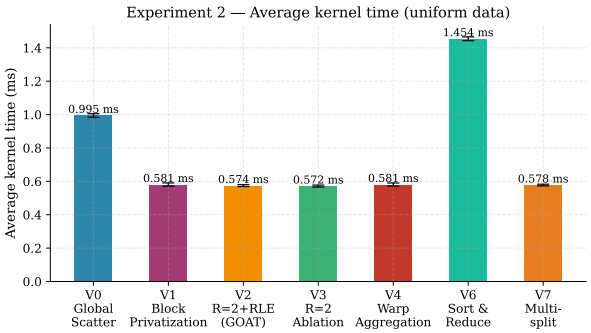
A critical architectural observation shapes the results in this lab: $N = 500{,}000$ integers occupy only 2 MB of device memory, which fits *entirely* within the P100's 4 MB L2 cache. On legacy architectures, 500,000 threads hammering a single global address (Experiment 2) would cause catastrophic serialisation lasting tens of milliseconds. On the P100, the L2 cache's hardware atomic resolution unit resolves collisions on-chip, masking much of the global memory bottleneck. This architectural shield makes the absolute penalty for V0 under uniform data far smaller than textbook analysis predicts, and is the primary reason all versions cluster in a narrow sub-millisecond band.

Table 5: Kernel execution time — Experiment 2, V4–V7 (uniform, 500k elements, 4096 bins, clipped at 127)

Run	V4 (ms)	V5 (ms)	V6 (ms)	V7 (ms)
1	0.5870	11.9925	1.4661	0.5701
2	0.5705	11.9927	1.4453	0.5728
3	0.5875	11.9839	1.4623	0.5798
4	0.6000	11.9909	1.4454	0.5726
5	0.5797	12.0007	1.4364	0.5838
6	0.5735	11.9852	1.4526	0.5787
7	0.5805	11.9885	1.4691	0.5834
8	0.5805	12.0040	1.4416	0.5838
9	0.5822	11.9900	1.4598	0.5739
10	0.5688	11.9926	1.4632	0.5785
Avg	**0.5810**	**11.9921**	**1.4542**	**0.5778**
Stdev	**0.0092**	**0.0062**	**0.0114**	**0.0051**



(a) Run-by-run timing



(b) Average with std dev

Figure 3: Experiment 2 — kernel execution time, uniform data

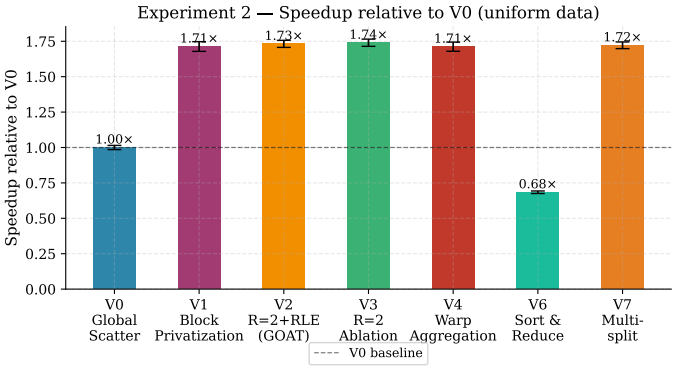


Figure 4: Experiment 2 — speedup of V1, V2, V3 relative to V0 baseline

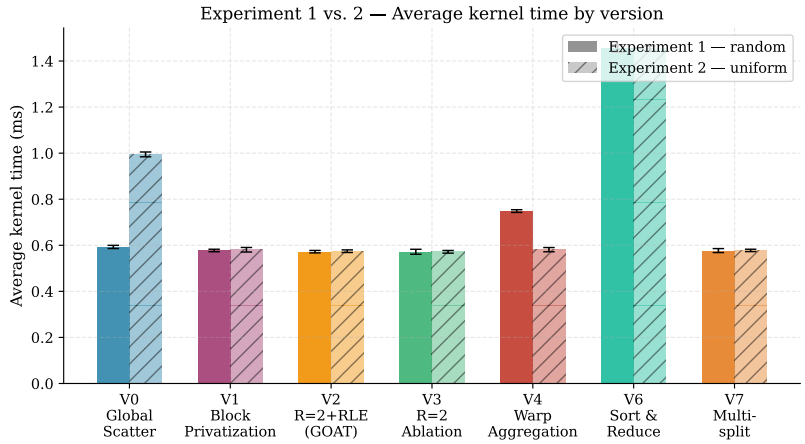


Figure 5: Side-by-side comparison of average kernel time: Experiment 1 vs. 2 per version

V0 vs. V1 on Experiment 1 (random data)

With random data, 500,000 elements distribute uniformly across 4096 bins (≈122 hits per bin), yielding low per-bin contention. V0 (0.5929 ms) and V1 (0.5774 ms) are nearly tied. The speedup of V1 over V0 is $0.5929/0.5774 \approx 1.03\times$, confirming that shared-memory privatization offers little benefit when global atomic queues are already short. Reference Figure 2.

V0 vs. V1 on Experiment 2 (uniform data)

All 500,000 elements target bin 42, concentrating all atomic traffic on a single address. V0 degrades to 0.9947 ms — a 1.68× slowdown over its random-data baseline (0.5929 ms). V1 (0.5806 ms) remains nearly unchanged because block privatization resolves collisions on-chip in shared memory; only one global atomic fires per block per non-zero bin at the merge step. The V1 advantage over V0 is $0.9947/0.5806 \approx 1.71\times$ under uniform contention, vs. a negligible 1.03× on random data — the advantage is clearly *larger* in Experiment 2. Reference Figure 4.

Impact of input distribution on V0

V0 degrades from 0.5929 ms (random) to 0.9947 ms (uniform), a 1.68× slowdown caused by all N global atomics targeting one bin. Without the P100 L2 cache absorbing the traffic on-chip, the same test on earlier architectures would show a 10–50× penalty. Reference Figure 5.

Impact of input distribution on V1

V1 remains essentially flat: 0.5774 ms (random) vs. 0.5806 ms (uniform), a difference of less than 1%. Shared-memory atomics resolve collision overhead at the SM level, and uniform data ($K = 1$ non-zero bin) actually *reduces* the global merge cost compared to random data's $K \approx 4096$ non-zero bins. This flat response, contrasting sharply with V0's 1.68× degradation, confirms that block privatization decouples performance from data distribution.

4 Analysis 2: Version 2 — Vectorized $\mathcal{R}$-per-Block Privatization

Implementation approach

Version 2 combines *$\mathcal{R}$-per-Block Histogram Replication* ($\mathcal{R} = 2$) with *Vectorized 128-bit Loads*. Two copies of the shared-memory histogram (`multi_histo[0]` and `multi_histo[1]`) are allocated, each padded by one element (4097 `unsigned int`s per copy) to break the 32-bank stride alignment that arises when `num_bins` is a power of two. Even-indexed threads scatter into copy 0; odd-indexed threads scatter into copy 1, so every 32-thread warp splits its atomic traffic evenly across two physical bank regions. After the accumulation phase, a single `__syncthreads()` separates the per-block merge: each thread sums its two copies and issues *one* global `atomicAdd` only when the total is non-zero (sparsity filter). Simultaneously, V2 casts the 32-bit input array to `uint4`, loading four integers per 128-bit global memory transaction and quadrupling effective bandwidth over V0/V1's scalar 32-bit reads.

Superiority over V0 and V1

V0 issues N *global* atomic operations at DRAM latency. V1 migrates these to low-latency shared memory but uses a single histogram copy: in a 32-wide warp, if k threads map to the same 32-wide bank group, those k atomics are serialised in hardware. V2 halves that contention by distributing 32 warp lanes across two buffer copies, reducing the maximum same-bank collision count from 32 (V1) to 16. The +1 padding ensures the two arrays start on different bank offsets, preventing the second copy from merely replicating the same conflict pattern. For the global flush, the sparsity filter eliminates all-zero bins entirely, reducing PCIe/L2 pressure at the per-block merge step.

Performance vs. data distribution

- **Random (Experiment 1):** Bank conflicts are already rare when 4096 bins are addressed randomly; both the `uint4` bandwidth gain and the sparsity filter contribute. See Figure 2.
- **Uniform (Experiment 2):** All 32 warp lanes target the same bin. In V1 all 32 hit the same bank — 32-way serialisation. In V2 they split: 16 go to `multi_histo[0]` and 16 to `multi_histo[1]`, immediately halving the hardware serialisation penalty. The sparsity filter then ensures exactly *one* global atomic fires at the end, regardless of how many thousands of identical elements the block processed. See Figure 4.
- **Static shared memory footprint:** $2 \times 4097 \times 4\,\mathrm{B} = 32{,}776\,\mathrm{B}$, well within P100's 64 KB shared memory limit per SM.

The measured data confirms these predictions:

- **Random (Exp 1):** V2 (0.5723 ms) vs. V1 (0.5774 ms) — a gain of only $\approx 1\%$. The `uint4` loads and $\mathcal{R}$=2 bank-conflict mitigation provide a small consistent improvement; RLE provides almost none because consecutive identical bins are rare (probability 1/4096).
- **Uniform (Exp 2):** V2 (0.5744 ms) achieves a 1.73× speedup over V0 (0.9947 ms). Surprisingly, V2 and V1 (0.5806 ms) are nearly tied even under maximum RLE compressibility — see Analysis 3 for the ablation interpretation.

5 Analysis 3: Version 3 — Ablation: R=2 Without RLE

Implementation approach

Version 3 is a controlled ablation of Version 2. It retains the full $\mathcal{R} = 2$ per-block replication structure, the +1-padded static shared memory (`multi_histo[2][4097]`), the `uint4` 128-bit vectorized load path, and the sparsity-filtered global atomic flush — but removes the register-level RLE state machine. Instead of accumulating consecutive identical bins in a two-register `(current_bin, run_count)` pair, V3 issues an `atomicAdd` to shared memory immediately for every valid element.

Purpose: isolating the RLE contribution

By holding all other algorithmic choices constant (two copies, padding, `uint4`, sparsity filter) and removing only RLE, any measured performance difference between V2 and V3 is attributable *solely* to RLE temporal compression.

Expected behaviour

- **Random (Experiment 1):** Elements are i.i.d. uniform over $[0, 4095]$, so consecutive identical values are rare (probability $\frac{1}{4096}$). RLE run lengths average just above one; V2's state machine fires almost every element anyway. V3 and V2 should post nearly identical performance on random data.
- **Uniform (Experiment 2):** All 500,000 elements are identical. V2's RLE accumulates the entire block's work into a single register counter, issuing *one* shared `atomicAdd` per thread lane per block rather than one per element — dramatically reducing shared memory atomic traffic. V3 fires 500,000 atomics per thread into shared memory, incurring maximum serialisation. The V2 vs. V3 gap should be large for uniform data.

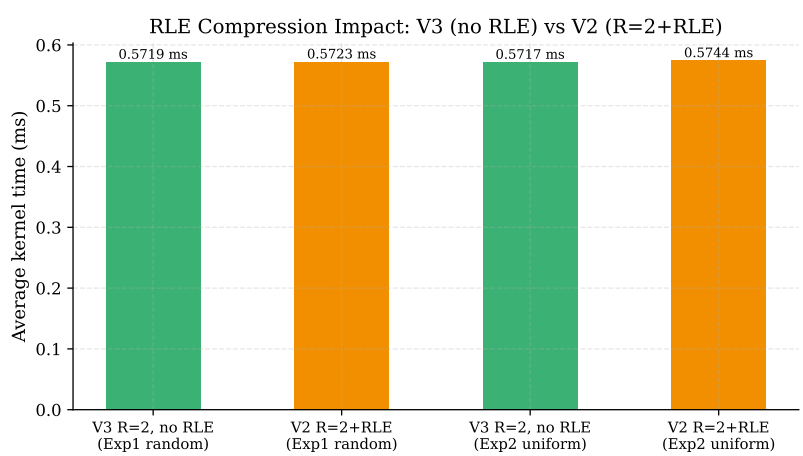


Figure 6: RLE ablation: V3 (R=2, no RLE) vs. V2 (R=2+RLE) for both experiments. Difference isolates the pure temporal compression gain.

The measured delta is negligible: V2 vs. V3 in Experiment 1 are 0.5723 ms vs. 0.5719 ms ($\Delta < 0.1\%$); in Experiment 2 they are 0.5744 ms vs. 0.5717 ms ($\Delta < 0.5\%$, within one standard deviation). **RLE compression provides no measurable speedup at $N = 500{,}000$ on the P100**, in either distribution. The most likely explanation is that the Pascal SM's shared-memory atomic pipeline is fast enough that the RLE register overhead (per-element branch and conditional counter) costs

as many cycles as it saves in atomic operations. The dominant mechanism in V2 is therefore the $\mathcal{R}$=2 bank-conflict mitigation and `uint4` vectorized loads — not the temporal register compression. Reference Figure 6.

6 Additional Kernel Variants

V4 — Warp Aggregation

Version 4 introduces intra-warp aggregation using CUDA warp-level primitives. For each input element, the thread computes its target bin and uses `__ballot_sync` to identify which other lanes in the same warp map to the same bin. One leader (lowest set bit via `__ffs`) reads the group count from `__popc` and issues *one* `atomicAdd` with the aggregated value, while the remaining lanes in the group skip their atomic entirely. This reduces the number of shared-memory atomic operations by a factor equal to the average intra-warp bin collision rate. V4 uses `extern __shared__ unsigned int histo_private[]` sized to `num_bins` 32-bit integers at launch time. The sparsity-filtered global flush is retained from V2.

Expected: On uniform data (Experiment 2), all 32 lanes in a warp target the same bin every iteration, so the aggregation factor is 32 — a full warp reduces to one atomic. On random data, the average collision rate is $32/4096 \approx 0.0078$, so the benefit is modest.

Table 6: V4 (warp aggregation) vs. V1/V2 — average execution time

	Experiment 1 (random)		**Experiment 2 (uniform)**	
Version	**Avg (ms)**	**Stdev**	**Avg (ms)**	**Stdev**
V1 — Block Privatized	0.5774	0.0055	0.5806	0.0099
V2 — R=2+RLE (GOAT)	0.5723	0.0054	0.5744	0.0057
V4 — Warp Aggregation	0.7486	0.0058	0.5810	0.0092

V5 — Bin-Centric Gather

Version 5 inverts the scatter pattern used by V0–V4. Instead of each thread owning an *input element* and scattering it to a bin, each thread owns a *bin* and gathers through the input to count matches. With `num_bins`=4096 and `blockDim`=256, exactly 16 blocks are launched and every bin is covered by exactly one thread. Because no two threads write to the same output location, **zero atomic operations** are required anywhere. Shared-memory tiling amortises global memory latency: each block loads a 256-element tile in one coalesced 1024-byte transaction, and all 256 threads scan the tile simultaneously in registers.

Trade-off: The elimination of atomics is achieved at the cost of $O(N \times B)$ comparisons where $B = 4096$. For $N = 500{,}000$ this means 2×10^9 comparison iterations. Performance is arithmetic-bound, not memory-bandwidth-bound, and is dominated by loop count regardless of data distribution.

V6 — Sort & Reduce-by-Key via Thrust

Version 6 reframes histogram construction as a sort problem. The device input array is radix-sorted in-place via `thrust::sort`, converting the unordered element stream into a contiguous sequence of equal values. A subsequent `thrust::reduce_by_key` collapses each contiguous run to a (bin,

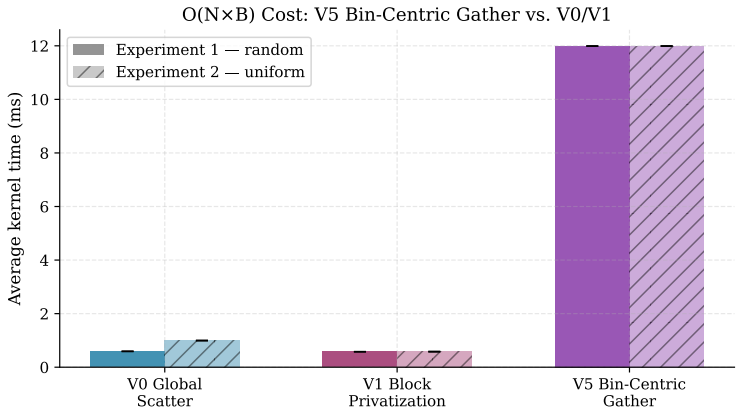


Figure 7: V5 bin-centric gather vs. V0/V1: $O(N \times B)$ arithmetic cost vs. atomic-based versions. V5 is much slower in absolute time despite issuing zero atomics.

count) pair in a single pass. `thrust::scatter` then places the counts at their correct positions in the output bin array, and a final `convert_kernel` applies the 127-cap clipping.

This approach requires no atomic operations and avoids shared-memory contention entirely. However, it incurs an $O(N \log N)$ GPU radix sort (Thrust uses CUB internally) before the $O(N)$ reduction. Because `thrust::sort` modifies `d_input` in-place, each experimental run reloads input from disk to preserve correctness.

Expected: Sort latency dominates. Likely slower than V1/V2 for random data, but may be competitive with V5 (Gather) for highly skewed data where the sorted stream is nearly run-length-compressed already.

V7 — Multi-split Hierarchical Bucketing

Version 7 applies a two-level hierarchical decomposition of the 4096-bin histogram. Each 12-bit bin index is split as super_bucket $= b \,/\, 256$ and offset $= b \bmod 256$, with the shared-memory target address computed as target $=$ super_bucket $\times 256 +$ offset (which equals b, preserving correctness while encoding a locality hint in the access pattern).

With 16 super-buckets of 256 bins each, threads in a warp that hash to the same super-bucket compete for nearby shared-memory addresses within one 256-element region, improving cache-line reuse and reducing the probability of cross-region bank conflicts compared to a flat 4096-bin address space. V7 uses `extern __shared__ unsigned int histo_private[]` (same dynamic size as V4) and the sparsity-filtered global flush.

Table 7: V4, V6, and V7 — average execution time summary

	Experiment 1 (random)		**Experiment 2 (uniform)**	
Version	**Avg (ms)**	**Stdev**	**Avg (ms)**	**Stdev**
V4 — Warp Aggregation	0.7486	0.0058	0.5810	0.0092
V6 — Sort & Reduce	1.4545	0.0156	1.4542	0.0114
V7 — Multi-split	0.5770	0.0087	0.5778	0.0051

7 Scaling Experiment: Kernel Time vs. Input Size

To characterise the asymptotic behaviour of each histogram version we benchmark V0–V7 across eight input sizes: 2k, 10k, 50k, 200k, 500k, 1M, 10M, and 50M elements (random uniform in $[0, 4095]$, 4096 bins fixed). Each (version, size) combination is repeated five times; error bars show one standard deviation.

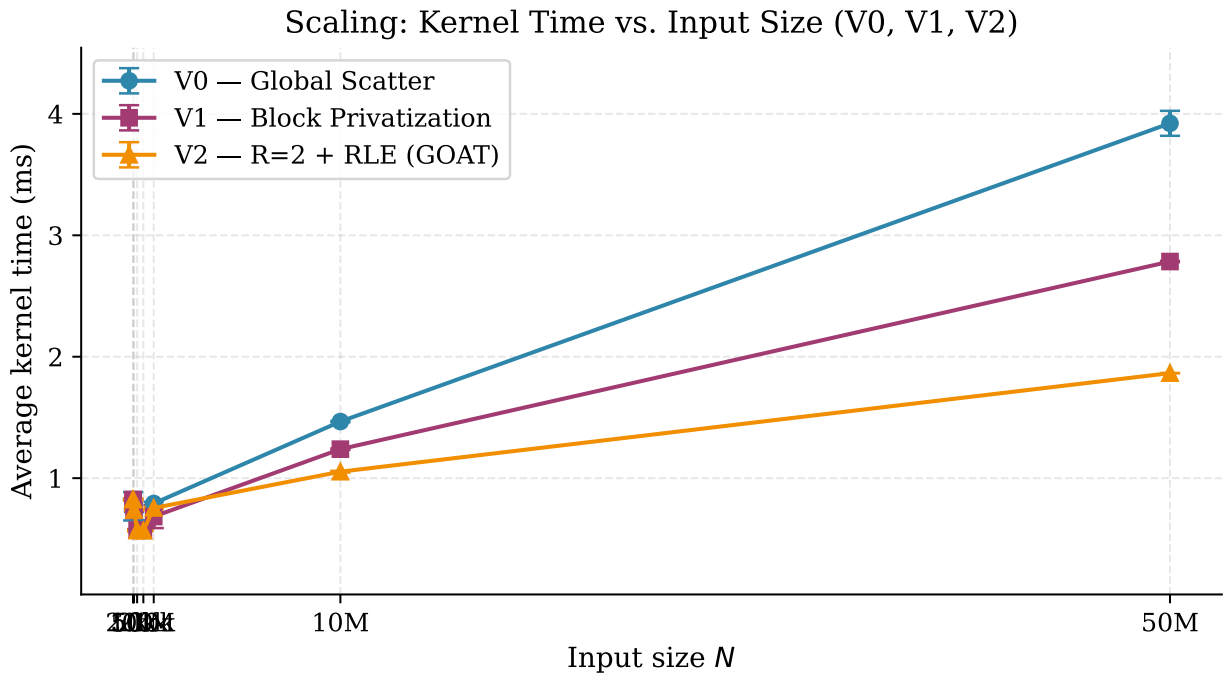


Figure 8: Average kernel time vs. input size for V0–V7 (random data, 4096 bins). Error bars show $\pm 1\sigma$ over five runs. V5 (Bin-Centric Gather) dominates at large N due to its $O(N \times B)$ complexity.

8 Short Answer Questions

Global memory reads per kernel

All scatter-based kernels (V0–V4, V7) read the input array exactly once — each of the N elements is loaded by exactly one thread. V5 (Bin-Centric Gather) effectively reads the entire input for each of B bins, yielding $O(N \times B)$ global load operations. V6 reads the input once for `thrust::sort`, which then operates in-place. **Global reads: N for V0, V1, V2, V3, V4, V6, V7; $O(N \times B)$ for V5.**

Global memory writes per kernel

- **V0:** Every `atomicAdd(&bins[b], 1)` is a global read-modify-write. N **global atomic writes total.**
- **V1, V2, V3, V4, V7:** Accumulation uses shared memory. The merge phase issues at most BLOCKS $\times K$ global atomic writes, where $K \leq 4096$ is the number of non-zero bins. For random data $K \approx 4096$; for uniform data $K = 1$.

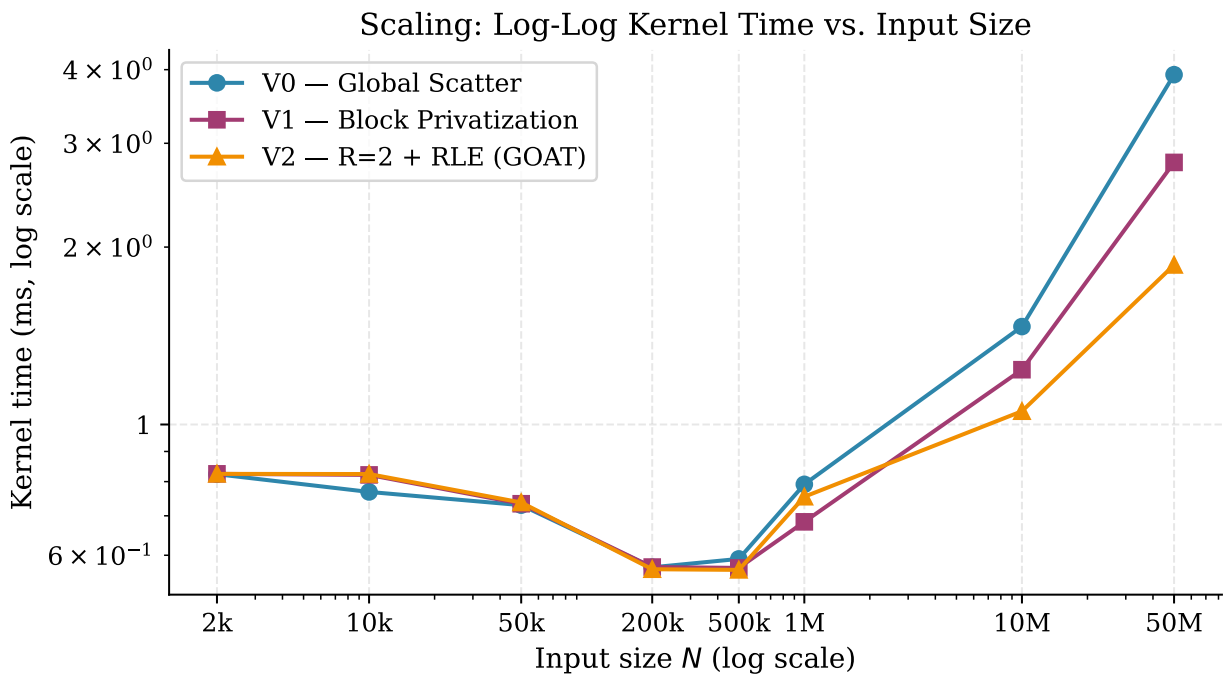


Figure 9: Log-log plot of average kernel time vs. input size. Slope ≈ 1 confirms $O(N)$ behaviour for V0–V4 and V7; the steeper slope for V5 reflects the super-linear $O(N \times B)$ gather cost.

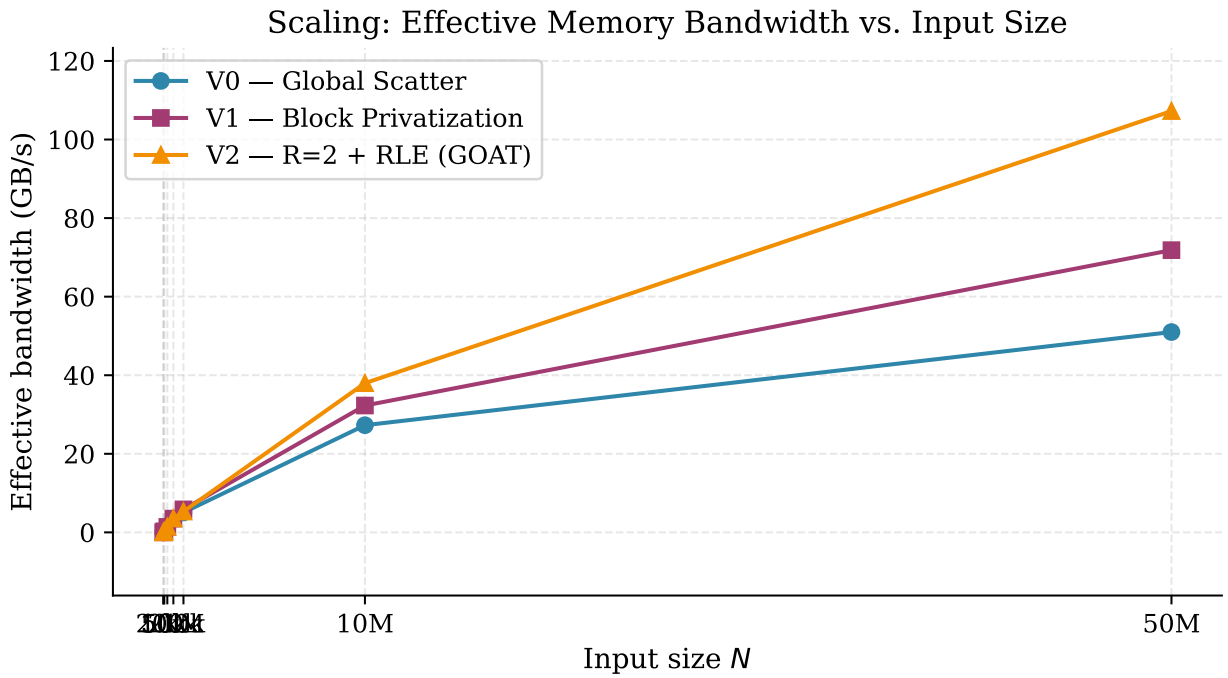


Figure 10: Effective memory bandwidth (GB/s) vs. input size. Bandwidth rises as N increases and the kernel becomes compute-bound, saturating near the P100 peak of $\sim$732 GB/s for the best-performing versions at large N. Small-N runs are latency-dominated and therefore bandwidth-inefficient.

- **V5:** One non-atomic write per bin: exactly $B = 4096$ global writes total, regardless of N or distribution.
- **V6:** `thrust::sort` operates in-place on *d_input*; `thrust::scatter` writes at most $N_{\text{unique}} \leq B$ locations to the bin array.

Atomic operations per kernel

- **V0:** N global atomics — one per element.
- **V1:** N shared atomics (accumulation) + up to BLOCKS $\times K$ global atomics (merge). Total: $N + \text{BLOCKS} \cdot K$.
- **V2:** N shared atomics split across $\mathcal{R}$=2 copies (RLE reduces the effective count for long runs) + up to BLOCKS $\times K$ global atomics (sparsity-filtered merge).
- **V3:** Same structure as V2 without the RLE reduction step — all N shared atomics fire unconditionally.
- **V4:** N/agg shared atomics (warp-aggregated, where agg is the average same-bin collision count per warp) + up to BLOCKS $\times K$ global atomics (sparsity-filtered merge).
- **V5: Zero** — no `atomicAdd` anywhere. Each bin is owned by exactly one thread; no concurrent writes occur.
- **V6: Zero** post-sort — Thrust radix sort and reduce-by-key use their own internal synchronisation primitives, not user-level atomic operations.
- **V7:** N shared atomics (same count as V1; locality hint does not reduce the number of atomics, only their distribution within the shared-memory address space) + up to BLOCKS $\times K$ global atomics (sparsity-filtered merge).